NET (BMEVITMAC09), midterm summary

Thursday 2026-10-29, 8:15, IE218: 6 open questions, pass at 12/30. Sessions 1 to 6, sections ranked by likelihood based on the 2018 midterm and the 2020 exams. On the paper: state the rule, apply it, conclude; a correct justification earns points even if the calculation goes wrong.

1. IPv4 addressing: netmask, network, same subnet

1 bits in the octet	1	2	3	4	5	6	7	8
Netmask octet	128	192	224	240	248	252	254	255
Magic number m	128	64	32	16	8	4	2	1
In the 4th octet	/25	/26	/27	/28	/29	/30		
Hosts	126	62	30	14	6	2		

Weights: 128, 64, 32, 16, 8, 4, 2, 1. /n: 2^{32-n} addresses, $2^{32-n}-2$ hosts (network and broadcast removed). $n = 8+8+\cdots+r$: as many 255 octets as there are 8s, then the octet with r bits (/19 = 255.255.224.0; 255.224.0.0 = /11).

Method: network, broadcast, same subnet.

1. Validate: octets ≤ 255; netmask = all 1s *then* all 0s (255.96.0.0 is invalid).
2. Interesting octet (netmask neither 255 nor 0); $m = 256$ minus the netmask octet.
3. Network = largest multiple of $m \leq$ the octet value, octets to the right set to 0 (= address AND netmask).
4. Broadcast = network $+m-1$, octets to the right set to 255. Hosts from network +1 to broadcast −1.
5. Same subnet $\iff$ same first n bits $\iff$ same network address.

Past exam examples. 192.168.115.29/18 and 192.168.78.247/18 (2018 midterm): 3rd octet, $m = 64$; 115 = 01110011 and 78 = 01001110 both start with 01, both in 64 to 127: **yes**, 192.168.64.0/18. 192.168.100.33 and .233, netmask 255.255.255.192 (/26): 33 gives .0, 233 gives .192: **no**. 192.168.79.178 and 192.168.178.79, netmask 255.255.192.0 (2020): 64.0 and 128.0: **no**. Slide: 152.66.163.9/18 gives 152.66.128.0, broadcast 152.66.191.255, 16,382 hosts.

Pitfalls. A different 3rd octet proves nothing in a /18; three identical octets are not enough in a /26. Always 8 bits: 224 = 11100000 (the slide writes 1110000). 266 is not an octet. Network and broadcast addresses cannot be assigned (193.225.14.63/28 is a broadcast). A route must be aligned: 10.0.20.0/16 is not a network (it is 10.0.0.0/16).

Classes and special addresses.

Class	First bits	First octet	Netmask, use
A	0	1 to 126	/8; 126 networks, $2^{24}-2$ hosts
B	10	128 to 191	/16; 2^{14} networks, 65,534 hosts
C	110	192 to 223	/24; 2^{21} networks, 254 hosts
D	1110	224 to 239	multicast
E	11110	240 and above	reserved

127: loopback. Private (not routable, NAT): 10.0.0.0/8, 172.16.0.0/12 (172.16 to 172.31), 192.168.0.0/16. Host part all 0: network; all 1: broadcast. 0.0.0.0/0: default route. 255.255.255.255: limited broadcast (DHCP Discover). The slide writes 10.0.0.0/24 and 127.0.0.0/24: typo, it is /8.

Assignable address before CIDR (2020 multiple choice): octets ≤ 255; class A, B or C from the first octet; class netmask; host part neither all 0 nor all 1. 191.48.255.0 and 190.13.254.255: yes (class B, the host part is two octets). 223.46.255.0 (C network), 223.46.253.255 (C broadcast), 224.43.57.24 (D), 187.258.22.57: no.

Subnetting. s borrowed bits: 2^s subnets $/(n+s)$, the k-th one at base $+k \times 2^{32-(n+s)}$. For H hosts: smallest h with $2^h - 2 \geq H$, prefix $32-h$ (300 gives /23, 62 gives /26, 63 gives /25). VLSM: needs in decreasing order, smallest block that fits, allocate one after the other (alignment follows), check the total. 2019 practice session: 131.108.64.0/20 for A (≤ 2000), B and C (≤ 1000): A 131.108.64.0/21, B 131.108.72.0/22, C 131.108.76.0/22, one OSPF area per unit, a single /20 advertisement outside.

2. Routing table: longest prefix match, aggregation, ARP

Longest prefix match. (1) Range of each line: is the destination in it? (2) Among the matching lines, the longest wins. (3) 0.0.0.0/0 only if nothing else matches. 2018 midterm, version I: 128.192.129.0/25 IF1, 128.192.128.0/17 IF2, 128.0.0.0/8 IF3, destination 128.192.129.254. The /25 covers .129.0 to .129.127 (254 starts with 1): no. The /17 (128.192.128.0 to 128.192.255.255) and the /8 match: **IF2**. Version II: 128.192.128.0/25, 128.192.0.0/17, 128.0.0.0/8, destination 128.192.128.254: the /17 only covers 128.192.0.0 to 128.192.127.255, only the /8 remains: **IF3**. Pitfall: a line that "starts like" the destination does not necessarily contain it.

CIDR aggregation. (1) Sort, write the ranges. (2) Merge two blocks only if they are contiguous, of the same size, toward the same next hop, and if the result is aligned (only the last bit of the prefix differs; start is a multiple of the new size). (3) Repeat. (4) Check that the result covers exactly the same addresses.

2018 midterm, I.10 (lecturer's solution, 2019 practice session): toward I1, 194.100.0.0/24, 1.0/24, 2.0/23, 4.0/22, 8.0/21, 48.0/20, 64.0/20. 0/24 + 1/24 = 0/23, + 2/23 = 0/22, + 4/22 = 0/21, + 8/21 = **0.0/20** (0.0 to 15.255). 48/20 is not contiguous (16 to 47 are missing). 48/20 and 64/20 are contiguous but not a /19: 48 = 00110000, 64 = 01000000, a /19 starts at a multiple of 32. Result: **three lines** 194.100.0.0/20, 194.100.48.0/20, 194.100.64.0/20 (version II: same table in 172.22). 2019 practice session: 200.0.0.0/18, 200.0.64.0/18, 200.0.128.0/17, 200.1.0.0/16 toward A give 200.0.0.0/15; 193.0.2.0 to 193.0.5.0 (/24) toward B give 193.0.2.0/23 and 193.0.4.0/23 (2 and 4 differ in bit 22, no /22). Aggregating wider would capture absent destinations.

CIDR allocation (2019 practice session): block = power of 2 $\geq$ need, aligned; group by interface. 194.48.0.0/16; A 2048 and C 4096 on IF1, B 8192 on IF2: A 194.48.0.0/21, reserve 194.48.8.0/21, C 194.48.16.0/20 (IF1), B 194.48.32.0/19 (IF2). Three lines; two if the reserve also goes to IF1 (194.48.0.0/19).

Interface addresses, tables. (1) One IP network per link, a /30 between two routers. (2) A free address of that network for each interface; a PC's gateway = the router interface on its LAN. (3) Table: directly connected networks, remote networks via the neighbor's address *on the shared network*, 0.0.0.0/0 toward the provider. (4) Routes in both directions; private addresses toward the Internet: NAT on the public interface. 2018 midterm: PC A 192.168.1.10/24, GW Eth1 192.168.1.1/24, GW Eth0 10.0.0.1/30, R2 Eth1 10.0.0.2/30, R2 Eth0 172.16.1.1/24, PC B 172.16.1.10/24; GW: 172.16.1.0/24 via 10.0.0.2; R2: 192.168.1.0/24 via 10.0.0.1. 2019 practice pitfall: the next hop is 128.96.34.130 (R1 on the shared network), not 128.96.34.1.

Path of a packet with ARP (both versions of the 2018 midterm).

1. The sender compares the destination with its own network (address AND netmask): same network, MAC of the destination; otherwise, MAC of the gateway.
2. Unknown MAC: ARP request broadcast (FF:FF:FF:FF:FF:FF) "who has IP?", unicast reply, cached.
3. Frame from the sender's MAC to the next hop's MAC; IP packet from source IP to destination IP.
4. Each router: longest prefix match, ARP on the next link, new frame; IP addresses unchanged, TTL −1.
5. Symmetric return; the ARP caches are already filled.

To write: the IP addresses stay the same end to end, the MACs change on every link; the switch is transparent (it learns source MACs). ARP has no authentication: cache poisoning.

3. Dynamic routing

Forwarding: choosing the next hop, packet by packet, with the table. **Routing**: discovering the topology (the protocol that fills the table). FIB: forwarding, fast lookup; RIB: discovery. Static or dynamic; flat or hierarchical (intra-domain and inter-domain). Loops: TTL decremented, packet dropped at 0. 1:1 redundancy (backup) or 1+1 (two copies), useful only if the paths are disjoint.

Source routing or hop-by-hop (2018 midterm, 2020).

	Source routing	Hop-by-hop
Principle	the source discovers the topology and writes the path into the packet	each router chooses the next hop with its table (IP)
Pro	the source chooses the path; simple routers	source needs no topology; local adaptation to failures; short header
Con	topology must be known; trust in the source (not used on the Internet, used in ad hoc sensor networks); fixed path; long header	no control by the source; loops if tables are inconsistent

Pitfall: do not confuse the local decision (forwarding) with the routing protocol.

Distance Vector and Link State.

	Distance Vector (DV)	Link State (LS)
Slogan	tell my neighbors who I know	tell everybody I know who my neighbors are
Sent	(destination, distance), without next hop, to the neighbors	my links and their cost (LSA) to all
Computation	distributed Bellman-Ford	Dijkstra on the full topology
Convergence	slow, one more hop per round; count to infinity	faster after a failure
Examples	RIP, IGRP	OSPF, IS-IS

Bellman-Ford: $d_X(Y) = \min_v \left(c(X,v) + d_v(Y)\right)$, v neighbor of X; $c = 1$ in RIP. Runtimes (slide): Dijkstra $O(|E| + |V| \log |V|)$, Bellman-Ford $O(|V|^2)$.

Count to infinity (6 exams, including the 2018 midterm). A DV advertisement does not contain the path: after a failure, a router adopts a neighbor's route that goes through itself. The two keep bouncing the destination back with an increasing distance up to RIP's infinity, 16 (hop count limit 15); meanwhile packets loop. **Example.** A, B, C in a line, costs 1: B reaches C at 1, A at 2 via B. The B-C link goes down; before advertising the failure, B receives "C at 2" from A and switches to 3 via A; A switches to 4 via B; B to 5; … up to 16 (15 advertisements). **Fixes.** Split horizon: do not advertise a route to the neighbor that is its next hop. Poisoned reverse: advertise it to that neighbor with infinite cost. Infinity at 16: bounds the duration. Triggered updates (general reminder): advertise the failure immediately, not at the 30 s update. Limitation: split horizon and poisoned reverse only fix loops between two neighbors. For 3/3: principle, numerical example, one fix named and explained.

Dijkstra step by step. (1) Source at 0, others at ∞. (2) Get the node with the smallest distance from the queue: final. (3) For each non-final neighbor, if distance of the removed node + weight < known distance, replace it and record the predecessor. (4) Until the queue is empty; the first hop is read by following the predecessors back. Slide graph: A-B 6, A-D 2, B-C 2, B-E 1, C-F 2, C-G 5, D-E 2, E-F 4, F-G 1; source C. Cell "d,p" (distance, predecessor); bold: final.

Step	Removed	A	B	D	E	F	G
0	C (0)	∞	2,C	∞	∞	2,C	5,C
1	B (2)	8,B	**2**	∞	3,B	2,C	5,C
2	F (2)	8,B		∞	3,B	**2**	3,F
3	E (3)	8,B		5,E	**3**		3,F
4	G (3)	8,B		5,E			**3**
5	D (5)	7,D		**5**			
6	A (7)	**7**					

First hops from C: B for A (7, C-B-E-D-A), B (2), D (5), E (3); F for F (2) and G (3). Pitfalls: A does not use the direct B-A link (7 < 8); a removed node is never modified again; the FIB keeps only the first hop.

Protocols.

RIP	DV; metric: hop count, limit 15, 16 = infinity; update interval 30 s; RIPv2: CIDR, authentication; RIPng: IPv6
OSPF	LS; path cost (bandwidth relative to a reference); Hello for neighbors, LSA; areas, LSAs stay within the area, backbone 0.0.0.0 (identifier, not an IP address); on Ethernet, Designated Router and Backup DR
IS-IS	LS, not tied to IP, no backbone area, Internet provider networks
BGP	between ASes (EGP); path vector: whole paths; policy routing (money, trust); iBGP: full mesh, hence the Route Reflector

IGP within an AS (RIP, OSPF, IS-IS), EGP between ASes (BGP). Peering: ASes on the same level, not transitive; transit: a higher level AS connects lower level ASes; manual contracts. DR election (2020 multiple choice, 2020 slides): highest priority; on a tie, the highest router ID; priority 0: never DR; non-preemptive (a newcomer does not take over).

4. Multiple access and duplexing

Multiplexing: one source combines several streams (FDM, TDM, CDM). **Multiple access**: several independent sources share the channel (FDMA: one carrier per user; TDMA: one time slot per user, synchronization; CDMA: codes). **Duplexing**: both directions of a link (FDD: two frequencies; TDD: same frequency, taking turns; or one wire per direction). Channel arbitration: fixed allocation (frequencies in cellular), central entity on request (time slots in cellular), contention (Ethernet CSMA).

Method (2018 midterm, Q1 of both versions): separate who shares (between users) and which direction; define each; answer *yes* and describe the combination. The definition alone earned 1.5/3. **FDMA and TDD**: each user keeps its band; within that band, downlink period and uplink period alternate; synchronization and guard time; one channel per user instead of two with FDD. **TDMA and FDD**: two carriers (uplink, downlink), divided into frames and time slots; one slot on each (the GSM scheme).

	Fixed allocation	Contention
Example	carriers or time slots in cellular	Ethernet CSMA, ALOHA
Pro	no collision, guaranteed rate and delay	no central entity, fault tolerant, good at low load and bursty traffic
Con	capacity lost when the source is silent; vulnerable central entity	collisions, retransmissions, no delay guarantee; collapses at high load

ALOHA: send as soon as a frame is ready; without acknowledgement, collision, retransmission after a random delay. **Slotted ALOHA**: transmission only at the start of a slot, common clock; vulnerability from 2 frame times to 1, maximum throughput from $1/(2e) \approx 18.4\%$ to $1/e \approx 36.8\%$.

5. Transport: TCP, UDP, NAT

Ports	16 bits; well-known 0 to 1023 (IANA, superuser); ephemeral source port
Values	DHCP UDP 67 server, 68 client; DNS UDP/TCP 53; HTTP 80, HTTPS 443; SMTP 25, submission 465 (formerly 587); POP3 110, 995 TLS; IMAP 143, 993 TLS
Protocol (IP)	ICMP 1, IGMP 2, TCP 6, UDP 17, DCCP 33, SCTP 132
Conversation	5-tuple {IP 1, IP 2, Protocol, Port 1, Port 2}

UDP (RFC 768): connectionless, datagram-oriented, no reliability, no flow control or congestion control; adds only ports on top of IP. 8-octet header: Source Port, Destination Port, Length, Checksum (includes part of the IP header; 0 = no check, valid in IPv4, not anymore in IPv6). **TCP**: connection-oriented, bidirectional, stream-oriented, reliable, flow control and congestion control, no broadcast or multicast.

Connection setup. SYN (Sc); SYN, ACK (Ss, Sc+1); ACK (Sc+1, Ss+1). Random initial sequence numbers; numbers count octets, SYN and FIN consume one; ACK = next expected octet (100 octets sent from Sc+1: ACK Sc+101). Disconnect: FIN, ACK, often in four segments. SYN Flood (exhaust the table of half-open connections), defense: SYN Cookies.

TCP header (20 octets without options). Ports 16 + 16; Sequence 32; Acknowledgement 32; Data Offset 4 (32-bit words, 5 to 15); flags SYN, ACK, FIN, RST, PSH (push to the application), URG (with Urgent Pointer); Window Size 16 (free octets at the receiver); Checksum; TLV options: window scale (shift ≤ 14, window up to 1 GB), SACK, Timestamp (PAWS). Octet 13 (general reminder): CWR ECE URG ACK PSH RST SYN FIN, so 0x12 = SYN+ACK, 0x02 = SYN.

Reliability (2020): connection and initial sequence numbers; sequence numbers (order, duplicates); cumulative ACKs; retransmission on timeout, or on 3 duplicate ACKs (Reno); checksum; sliding window with flow control and congestion control.

Flow and congestion. **Flow control**: protects the *receiver*, Win-

dow Size = `adv_wnd`; at 0, the sender stops then probes with a small segment. **Congestion control**: protects the *network*, `cwnd` on the sender side. Effective window = min(`adv_wnd`, `cwnd`). Silly Window Syndrome: Nagle's algorithm on the sender side, window advertised only when half-empty on the receiver side. Congestion collapse: loss, retransmissions, load; goodput = useful share; stay left of the knee point, far from the cliff.

cwnd: rules. Unit MSS, one line per round trip (round), `cwnd` in segments.

- Start `cwnd` = 1, `ssthresh` = `adv_wnd`.
- **Slow start**: `cwnd` ×2 per round up to `ssthresh`, then CA.
- **Congestion avoidance**: `cwnd` +1 per round (AIMD), `ssthresh` follows `cwnd`.
- **Timeout** (Tahoe and Reno): `ssthresh` = `cwnd`/2 (half the window at the loss), `cwnd` = 1, slow start.
- **3 duplicate ACKs** (Reno): fast retransmit of the missing segment, then fast recovery: `cwnd`/2, stay in CA.

Convention to state on the paper: cwnd capped at ssthresh in slow start. If doubling exceeds `ssthresh`, stop at `ssthresh` (exercise convention of these documents; the slide only says "when cwnd reaches ssthresh").

Round	1	2	3	4	5 to 12	13	14	15	16	17	18
Tahoe, timeout in 12	1	2	4	8	9 to 16	1	2	4	8	9	10
Reno, 3 dup. ACKs in 12	1	2	4	8	9 to 16	8	9	10	11	12	13

Initial `ssthresh` 8; at the loss `cwnd` = 16, new `ssthresh` = 8. Total over 18 rounds: 149 segments (Tahoe), 178 (Reno). Cap: `ssthresh` 16, rounds 1 to 9: 1, 2, 4, 8, 16, 17, 18, 19, 20; timeout in 9: `ssthresh` 10, then 1, 2, 4, 8, **10** (16 capped), CA.

Fast retransmit (slide): segments 20 to 26, 21 lost. ACK 21 (for 20), then 21 for 22, 23, 24, 25; at the 3rd duplicate (for 24), 21 is retransmitted; when it arrives, ACK 26 (cumulative). Why 3: reordering without loss also produces duplicates.

Variants. Tahoe (1988): slow start and AIMD. Reno (1990): + fast retransmit and fast recovery. New Reno: retransmits on the first duplicate ACK, keeps `cwnd`. Vegas: congestion detected by delay, loses bandwidth to Reno. CUBIC: `cwnd` is a cubic function of the time since the last congestion event, mostly independent of the RTT; default in Linux, macOS, Windows.

Optimal window, BDP (3 exams in 2020). $W \geq$ rate × RTT; maximum throughput $\approx W/\text{RTT}$. (1) RTT = one way + way back. (2) Bottleneck rate. (3) Product in bits, /8. (4) Compare with 65,535 octets: above that, window scale option ($65{,}535 \times 2^s$, $s \leq 14$).

- RTT 100 ms, DSL 20 Mbit/s: $2 \cdot 10^6$ bit = **250 kB**; without window scale, $65{,}535 \times 8/0.1 \approx 5.2$ Mbit/s.
- 150 ms one way, 100 ms back, 16 Mbit/s: RTT 250 ms, **500 kB**.
- "Delay 200 ms", 8 Mbit/s: 400 kB if 200 ms each way, 200 kB if it is the RTT: write down the assumption.
- Lecture: 100 Mbit/s, RTT 80 ms: 10^6 octets; $65{,}535 \times 2^4 \geq 10^6$, $s = 4$.

Pitfalls: one-way delay only (125 kB would be wrong); bits and octets; the slide writes "64k * 1500", compute in octets.

Links with high loss rate (2019 practice session): BER 10^{-6}, 1500-octet packet = 12,000 bits, loss $\approx$ 1.2% (1 in 83.3). Reno treats the loss as congestion and shrinks its window; a fixed 20% error-correcting code (8 Mbit/s without loss) is usually better. Fixes: delay-based window scaling, fast retransmit, SACK.

2020 multiple choice. UDP guarantees nothing, TCP does; multicast only with UDP; TCP scales back on loss; congestion control protects the network, not the end host; UDP is used for real-time multimedia (a retransmission would arrive too late).

NAT. The gateway replaces the source IP with its public IP and the source port with an ephemeral port, and remembers the association; the reply is rewritten the other way.

Leg	Source	Destination
client to gateway	192.168.1.31:50000	152.66.244.17:443
gateway to server	94.44.112.4:40001	152.66.244.17:443
server to gateway	152.66.244.17:443	94.44.112.4:40001
gateway to client	152.66.244.17:443	192.168.1.31:50000

Outgoing connections only (S-NAT); an internal server needs D-NAT (or UPnP IGD). One connection = one distinct external port, otherwise two identical 5-tuples. Problems: limited number of ephemeral ports, peer-to-peer and H.323, signed endpoint addresses. DCCP: UDP with reliable connection setup and acknowledgement without retransmission. SCTP: datagrams, multihoming, telecom. QUIC and μTP over UDP (other sockets need special privileges).

6. XML to JSON and YAML (announced question, session 1)

"Possible exam question: turn the given XML into JSON/YAML" (`hta1p`, p. 26). (1) The root encloses everything: its content becomes the top-level object. (2) Text element: `"key": value` pair, number without quotes. (3) Element with different children: object `{}`. (4) Siblings with the same name: array `[]`, the repeated tag disappears. (5) YAML: same data, without braces or unnecessary quotes, one indentation level per nesting, one hyphen per list entry. (6) XML attributes: choose a convention and write it down. Slide example (p. 24 to 26, list shortened to two titles):

```
<NTALecture>
  <seq>2</seq>
  <title>Protocols Practical</title>
  <slides>
    <SlideTitle>Title</SlideTitle>
    <SlideTitle>Practical</SlideTitle>
  </slides>
</NTALecture>
```

```
seq: 2
title: Protocols Practical
slides:
  - Title
  - Practical
```

```
{
  "seq": 2,
  "title": "Protocols Practical",
  "slides": [ "Title",
              "Practical" ]
}
```

TLV: the length lets a reader skip an unknown type (extensible); DHCP encodes its options in TLV. YAML: superset of JSON, no schema validation, truncation cannot be detected. JSON: `JSON.parse()` rather than `eval()`. Network byte order is big endian (`htons`, `ntohl`).

7. IPv4: header, fragmentation, ICMP, DHCP, DNS

Bits	Field	Key points
4, 4	Version, IHL	4; IHL in 32-bit words, 5 = 20 octets (0x45)
6, 2	DSCP, ECN	service class; congestion signalled without dropping
16	Total Length	octets, header included, 20 to 65,535
16	Identification	shared by the fragments of a packet
3, 13	Flags, Offset	DF (drop instead of fragmenting), MF; offset in 8-octet units
8, 8	TTL, Protocol	loop avoidance; ICMP 1, TCP 6, UDP 17
16	Checksum	header only, recalculated at every hop (TTL)
32, 32	Source, Dest.	addresses; options if IHL > 5

IPv4: connectionless, best effort. Pitfall: three units (IHL ×4, octets, ×8). **Fragmentation.** Data per fragment $\lfloor(\text{MTU} - 20)/8\rfloor \times 8$; offset = octets already sent /8; MF = 1 except the last; same Identification. 4000 octets over MTU 1500: data 1480, 1480, 1020; offsets 0, 185, 370; MF 1, 1, 0. DF = 1: packet dropped, ICMP type 3 "fragmentation needed". Ethernet MTU 1500.

ICMP (Protocol 1; Type, Code, checksum): 0 Echo Reply, 3 Destination Unreachable (net, host, protocol, port, fragmentation needed), 5 Redirect, 8 Echo Request, 9 and 10 Router Advertisement and Solicitation, 11 Time Exceeded. Ping: Echo Request and Reply (latency, drop rate). Traceroute: TTL = 1, 2, 3…, each router where the TTL reaches 0 sends back Time Exceeded. Security: no authentication (fake Unreachable or Redirect), ping flood, ping tunnel, topology discovery; filtered by firewalls.

DHCP (UDP, server 67, client 68): DORA, Discover broadcast, Offer, Request broadcast (notifies all servers), Ack. TLV options: netmask, default gateway, DNS server. Lease = soft state, must be renewed. Others: decline, renew, release. **DNS**: name to IP; records A (IPv4), AAAA (IPv6), MX (email); UDP/53, TCP/53 for long messages. Distributed tree database; local resolver (given by DHCP); authoritative or cached response; reverse lookup. Weaknesses: no encryption, no authentication, cache poisoning, look-alike names.

8. Session 1: stack, QoS, statistical multiplexing

OSI	PDU	TCP/IP	Role
7, 6, 5	Data	Application	user; presentation (encoding); session
4 Transport	Segment	Transport	reliability, segmentation, multiplexing
3 Network	Packet	Network	forwarding, routing
2, 1	Frame, Bit	Link	neighbor; bits on the medium

Encapsulation: each layer adds its own header and only touches its own. SAP: access point to a layer's features (sockets). Planes: data (user traffic), control (connection, ACK; ICMP), management (configuration; routing now considered control). **Security**: encryption against eavesdropping (Diffie-Hellman), authentication and challenge-response against replay attacks, integrity by hashing with a secret; AAA. **QoS**: IntServ per flow, DiffServ per class, best effort on the Internet. Token bucket: $+K$ tokens/s, capacity B, metering (bursts possible). Leaky bucket: output $\leq K$, shaping, queue. K = average bitrate, $B = (\max - \text{avg}) \times$ burst length (2 and 10 Mbit/s, 0.5 s: $B = 4$ Mbit). RED: drop early, TCP slows down. **Statistical multiplexing**: $N = 500$, $p = 1\%$; average $Np = 5$; binomial approximated by Poisson $\lambda = Np$; blocking $\Pr(X > L)$; Chebyshev $\Pr(|X - \mu| \geq k\sigma) \leq 1/k^2$ (overestimates). Pitfall: the slide uses the variance as σ ($L = \lambda + 3\lambda = 20$); with $\sigma = \sqrt{5}$, $L \approx 11.7$. PCM: 8 kHz $\times$ 8 bits = 64 kbps.

9. Session 6: Web, TLS, QUIC, email (no past exams)

HTTP: request and reply, stateless. GET (object at the URL), POST (data sent), HEAD (metadata). Codes: 1XX in progress, 2XX success (200), 3XX redirection (301), 4XX client errors (403, 404), 5XX server errors (500, 502, 504). Headers User-Agent, Server, Referer; Host mandatory in 1.1. Versions: 1.0 (RFC 1945); 1.1 reuses the TCP connection; 2 (SPDY) header compression and request multiplexing; 3 over QUIC. Proxy: caching, load balancing, filtering. **Cookies**: key and value set by a response, sent back to the same site; session, personalization, tracking (third party cookies); a response with a cookie cannot be cached. **Crypto**: symmetric (shared secret; Diffie-Hellman, no protection against man-in-the-middle); asymmetric (encrypt with the receiver's public key; sign a hash with your own private key, SHA-256; RSA). Public keys: web of trust (PGP) or PKI (CA; X.509 certificate valid if issued to the site and signed by a trusted CA; self-signed: encryption without proving identity). **TLS**: SSL 3.0 then TLS 1.0, older than 1.2 deprecated; application layer; TLS handshake after the TCP handshake; StartTLS upgrades a connection to TLS. HTTPS: 443. **QUIC** (RFC 9000, HTTP/3): over UDP, in the application layer; transport and TLS handshake combined, 3-way instead of 6-way (1 RTT versus 2 for TCP + TLS 1.3); encryption is mandatory; transport session ID independent of ports (mobility); multiplexing aware of web elements.

Email: MUA → MSA (submission) → MTA (SMTP relay) → MDA (mailbox) → the recipient's MUA. The MUA sends to the MTA with SMTP, retrieves from the MDA with POP3 or IMAP; a domain's MTA is found with the MX record.

SMTP	25; 465 (formerly 587)	between MTAs and submission; MAIL, RCPT, DATA; codes like HTTP
POP3	110; 995	simple retrieval, RETR, DELE; inbox only
IMAP	143; 993	database on the server, folders, FETCH; browse without retrieving

RFC 5322 format: header fields (From, To, Subject…), empty line, body in 7-bit ASCII. MIME: convert everything to 7 bits (quoted-printable, base64: +1/3), Content-Type, multipart for attachments. Authentication: PLAIN inside TLS; CRAM (random number, hash(password, number), against replay attacks); OAuth 2.0; TOTP (secret and current time, 2FA). PGP: web of trust, sign and encrypt; S/MIME. Anti-spam: DKIM (the MTA signs, keys in DNS), SPF (MTA authorized for the domain in From), DMARC (policy published in DNS).